



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ**

Дипломна работа на тема:

**СИСТЕМА ЗА СЪЗДАВАНЕ, УПРАВЛЕНИЕ И СЪХРАНЕНИЕ НА
ДОКУМЕНТИ**

Изготвил:

Славян Овнарски фак. №121222038, 38 гр.

София, 2026

Съдържание

I. УВОД.....	5
II. ОПИСАНИЕ НА ПРОБЛЕМНАТА ОБЛАСТ	6
2.1. Контекст и мотивация	6
2.2. Съществуващи решения и техните ограничения	6
2.3. Цели на системата	8
2.4. Целева аудитория	8
2.5. Обхват и ограничения	9
2.6. Функционални изисквания към системата	10
2.7. Нефункционални изисквания към системата	11
III. ПРОЕКТИРАНЕ НА БАЗАТА ДАННИ И АРХИТЕКТУРАТА НА СИСТЕМАТА .	14
3.1. Системна архитектура - обзор.....	14
3.2. Компоненти на системата	16
3.2.1. Spring Boot API (MYDrive)	16
3.2.2. Go File Server.....	17
3.2.3. Next.js Frontend	18
3.3. Схема на базата данни.....	19
3.4. Комуникация между компонентите.....	20
3.5. Сигурност и автентикация	20
3.6. Инфраструктура - Docker и Nginx.....	21
3.6.1. Docker и Docker Compose	21
3.6.2. Nginx	21
IV. ОСОБЕНОСТИ НА ПРОГРАМНАТА РЕАЛИЗАЦИЯ.....	23
4.1. <i>Backend - Spring Boot</i>	23
4.1.1. Стартиране на приложението	23
4.1.2. Автентикация - <i>AuthController</i> и <i>AuthService</i>	23
4.1.3. Управление на файлове - <i>FileController</i> и <i>FileService</i>	25

4.1.4. Spring Security конфигурация.....	26
4.2. File Server - Go	27
4.3. Frontend - Next.js.....	27
4.3.1. Структура и маршрутизиране	27
4.3.2. AuthContext - глобално управление на автентикацията	28
4.3.3. API клиент (lib/api.ts)	28
4.3.4. DriveView - централен компонент на файловия мениджър	28
4.3.5. Next.js Proxy Rewrite.....	29
4.4. Реално-времево съвместно редактиране (WebSocket / STOMP).....	29
4.4.1. Сървърна конфигурация (WebSocketConfig)	29
4.4.2. CollaborationController	29
4.4.3. Клиентска страна (DocumentEditor).....	30
4.5. Електронно подписване на документи (eIDAS PAdES)	31
4.5.1. Автоматично генериране на demo keystore.....	31
4.5.2. Процес на PAdES-B-B подписване	31
4.6. AI Чатбот	33
4.7. Платежна интеграция и абонаменти (Stripe).....	34
4.8 Структура на база данни	36
4.9 Тестване на системата	39
4.9.1. Подход и инструменти	39
4.9.2. Покрити модули и сценарии.....	40
4.9.3. Примерен модулен тест	41
V. РЪКОВОДСТВО НА ПОТРЕБИТЕЛЯ.....	42
5.1. Регистрация и вход в системата	42
5.1.1. Регистрация на нов акаунт.....	42
5.1.2. Вход в системата.....	43
5.2. Управление на папки и файлове	44

5.2.1. Навигация	44
5.2.2. Създаване на нова папка	45
5.2.3. Преименуване	45
5.2.4. Изтриване	45
5.2.5. Местене на файл	45
5.3. Качване и изтегляне на файлове	46
5.3.1. Качване на файл.....	46
5.3.2. Изтегляне на файл	46
5.4. Споделяне на файлове и папки	46
5.4.1. Създаване на споделена връзка.....	46
5.4.2. Управление на споделяния	46
5.4.3. Достъп до споделена връзка	47
5.5. Съвместно редактиране на документи	47
5.5.1. Отваряне на документ за редактиране.....	47
5.5.2. Работа в редактора.....	47
5.6. Електронно подписване на PDF	47
5.6.1. Подписване на PDF документ	47
5.6.2. Преглед и изтегляне на подписания документ	48
5.7. AI Асистент	48
5.8. Управление на профила и абонамента	48
5.8.1. Промяна на парола	48
5.8.2. Информация за абонамента	48
5.8.3. Смяна на план и плащане	48
VI.ЗАКЛЮЧЕНИЕ	50
VII. СПИСЪК НА ИЗПОЛЗВАНИТЕ ЛИТЕРАТУРНИ ИЗТОЧНИЦИ	51
VIII. Приложение.....	52

I. УВОД

Настоящата дипломна работа е посветена на проектирането и реализацията на MyDrive – платформа за съхранение, управление и споделено редактиране на електронни документи. Основната цел на разработката е създаването на модерно приложение, което предоставя функционалностите на съвременните облачни решения, като същевременно осигурява по-висока степен на контрол върху данните и възможност за внедряване в инфраструктура, управлявана от организация.

В процеса на разработване са извършени анализ на съществуващите решения в областта на облачното съхранение на данни, дефиниране на функционалните и нефункционалните изисквания към системата и проектиране на многослойна архитектура, осигуряваща ясно разделение на отговорностите между отделните компоненти.

Приложението включва сървърна част, изградена със *Spring Boot*, файлов сървис за управление на физическото съхранение на файловете, уеб базиран потребителски интерфейс, релационна база данни за съхранение на метаданни и механизми за сигурна автентикация и авторизация. Допълнително са разработени функционалности за съвместно редактиране на документи в реално време, електронно подписване на *PDF* документи съгласно изискванията на регламента *eIDAS* и интеграция на асистент, базиран на технологии за изкуствен интелект.

Обяснителната записка е структурирана в четири глави. В първа глава са разгледани проблемната област, целите и обхватът на разработката. Втора глава представя архитектурата на системата, модела на базата данни и използваната инфраструктура. В трета глава е описана реализацията на отделните програмни компоненти. Четвърта глава съдържа ръководство за работа със платформата от гледна точка на крайния потребител. В заключението са обобщени постигнатите резултати, направените изводи и възможностите за бъдещо развитие на проекта.

II. ОПИСАНИЕ НА ПРОБЛЕМНАТА ОБЛАСТ

2.1. Контекст и мотивация

В условията на дигиталната трансформация организациите и индивидуалните потребители създават и обработват значителни количества електронни документи, включително договори, фактури, техническа документация, изображения и архивни данни. Ефективното управление и съхранение на тази информация е от съществено значение за осигуряване на непрекъснатост на работните процеси и защита на данните.

Традиционните подходи за съхранение на информация върху локални устройства са свързани с редица ограничения, като риск от загуба на данни при хардуерна повреда, затруднено проследяване на версиите на документите, ограничени възможности за съвместна работа и липса на централизирано управление на достъпа.

Съвременните облачни платформи за съхранение на файлове, като Google Drive, OneDrive и Dropbox, предлагат механизми за централизирано съхранение, споделяне и съвместна работа върху документи. Разликата при тези решения е управлението на инфраструктурата и съхранението на данните се осъществяват от външен доставчик, което може да създаде предизвикателства, свързани със сигурността на информацията, защитата на личните данни и спазването на нормативни изисквания, включително Общия регламент относно защитата на данните (GDPR).

В отговор на тези предизвикателства е разработен програмният продукт MyDrive – платформа за управление и съхранение на документи, предназначена за внедряване в инфраструктура, контролирана от организацията. Системата предоставя функционалности за съхранение и споделяне на файлове, съвместно редактиране на документи в реално време, управление на права за достъп и електронно подписване на документи съгласно изискванията на регламента *eIDAS* и стандарта *PAdES*. По този начин се цели съчетаване на удобството на облачните услуги с по-висока степен на контрол върху данните и съответствие с приложимите регулаторни изисквания.

2.2. Съществуващи решения и техните ограничения

При анализа на съществуващите решения за облачно съхранение и управление на файлове са разгледани няколко широко разпространени платформи. Google Drive е утвърдено решение, което предлага богат набор от функционалности и висока степен на интеграция с услугите на Google. Въпреки това съхранението на данните в

инфраструктурата на доставчика може да създаде предизвикателства, свързани със съответствието на регулаторни изисквания, включително тези на Общия регламент относно защитата на данните (*GDPR*).

Nextcloud представлява платформа с отворен код, която позволява внедряване в собствената ИТ инфраструктура на организацията. Това осигурява по-висока степен на контрол върху данните и по-голяма независимост от външни доставчици. От друга страна, внедряването и поддръжката на системата са свързани с по-голяма административна сложност и необходимост от допълнителни хардуерни ресурси.

Dropbox се отличава с лесен за употреба и интуитивен потребителски интерфейс, което улеснява работата на крайните потребители. В същото време платформата е със затворен код и може да доведе до по-високи разходи при внедряване в организационна среда.

SharePoint предоставя задълбочена интеграция с екосистемата на Microsoft 365 и предлага широк набор от инструменти за сътрудничество и управление на документи. Доставяйки тези предимства, платформата се характеризира с относително висока цена и необходимост от специализирани знания за нейната поддръжка.

При сравнение на разгледаните решения може да се обобщи, че всяко от тях поставя акцент върху различен аспект от управлението на документи. Google Drive и Dropbox се отличават с лесна употреба и бързо внедряване, но предоставят ограничен контрол върху инфраструктурата. Nextcloud осигурява по-голяма независимост, но изисква повече административни ресурси. SharePoint предлага богати корпоративни възможности, но е свързан с по-висока сложност и цена.

Това показва необходимостта от система, която съчетава удобството на съвременните облачни платформи с възможност за самостоятелно управление на данните. Именно в тази посока е насочена разработката на MyDrive, която цели да предостави основните функции за работа с документи, като същевременно запази гъвкавост при внедряване и бъдещо разширяване.

Разработваната платформа MyDrive има за цел да обедини ключовите предимства на разгледаните решения. Системата е проектирана да осигурява пълен контрол върху съхраняваните данни чрез внедряване в инфраструктура, управлявана от организацията, както и поддръжка на изискванията на регламента *eIDAS*. Допълнително са предвидени

функционалности за съвместно редактиране на документи в реално време и интегриран AI асистент, подпомагащ потребителите при ежедневната им работа.

2.3. Цели на системата

Основната цел на системата MyDrive е създаването на платформа за сигурно съхранение, управление и споделена работа с електронни документи. За постигането на тази цел са дефинирани следните основни задачи:

- Осигуряване на централизирано и защитено съхранение на файлове и папки.
- Изработване на механизми за сигурна автентикация, авторизация и управление на потребителски профили.
- Предоставяне на възможност за споделяне на файлове и папки чрез генерирани връзки с контрол на правата за достъп и ограничена валидност.
- Поддръжка на съвместно редактиране на документи в реално време от множество потребители.
- Осигуряване на възможност за електронно подписване на *PDF* документи в съответствие с изискванията на регламента *eIDAS*.
- Интегриране на интелигентен асистент, подпомагащ потребителите при работа с продукта.
- Реализиране на механизъм за управление на абонаментни планове и квоти за съхранение.
- Осигуряване на възможност за внедряване и експлоатация на системата в инфраструктура, управлявана от организацията, с цел по-висока степен на контрол върху данните.
- Гарантиране на мащабируемост, надеждност и възможност за бъдещо разширяване на функционалността на системата.

2.4. Целева аудитория

Системата MyDrive е предназначена за широк кръг потребители и организации, които ежедневно работят с електронни документи и се нуждаят от сигурно съхранение, споделяне и съвместна обработка на информация. Основните групи потребители са:

- Крайни потребители – служители в организации, студенти, преподаватели, юристи, счетоводители и други специалисти, които създават, редактират, споделят и съхраняват документи в рамките на своята професионална дейност. За тях

системата предоставя удобен достъп до файловете, възможност за съвместна работа в реално време и механизми за електронно подписване на документи.

- • Организации и екипи – компании, образователни институции и административни структури, които се нуждаят от централизирано управление на документи, контрол на достъпа до информацията и спазване на нормативни изисквания, свързани със защитата на данните.
- • Системни администратори – специалисти, отговорни за внедряването, конфигурирането и поддръжката на системата. Те управляват инфраструктурата, потребителските акаунти, политиките за сигурност и процесите по архивиране и възстановяване на данните.
- По този начин MyDrive е насочена както към индивидуални потребители, така и към организации, които желаят да използват съвременна платформа за управление на документи, като същевременно запазят висока степен на контрол върху своите данни и инфраструктура.

2.5. Обхват и ограничения

Настоящата версия на системата MyDrive е реализирана като уеб базирано приложение, достъпно чрез съвременни интернет браузъри. За целите на демонстрацията и тестването е използван тестов самоподписан сертификат при реализиране на функционалността за електронно подписване на документи. При внедряване в реална експлоатационна среда е необходимо използването на удостоверителни средства и сертификати, издадени от квалифициран доставчик на удостоверителни услуги, в съответствие с изискванията на регламента *eIDAS*.

Софтуерът поддържа управление на абонаментни планове и ограничения върху използваното дисково пространство.

Настоящата реализация на MyDrive обхваща основните процеси, свързани със съхранение, организация, споделяне и обработка на електронни документи в уеб среда. Системата е насочена към предоставяне на централизирано файлово пространство, в което потребителите могат да управляват своите документи чрез браузър, без необходимост от инсталиране на допълнителен клиентски софтуер.

В обхвата на разработката са включени функционалности за регистрация и вход в системата, управление на файлове и папки, споделяне чрез връзки, контрол на правата

за достъп, съвместно редактиране на документи, електронно подписване на PDF файлове и управление на абонаментни планове. Реализирана е и възможност за интеграция с AI асистент, който подпомага потребителите при работа със системата.

Извън обхвата на настоящата версия остават някои функционалности, характерни за големи корпоративни платформи за управление на документи. Такива са например детайлното версионизиране на всеки документ, сложни работни потоци за одобрение, разширени административни политики, автоматизирана класификация на съдържанието и пълна интеграция с външни офис пакети. Тези възможности могат да бъдат разглеждани като насока за бъдещо развитие на системата.

Ограничение на текущата реализация е, че тя е разработена основно с демонстрационна и учебна цел. Поради това някои механизми, като електронното подписване, са реализирани чрез тестови сертификати и не заместват напълно продукционна среда с квалифицирани удостоверителни услуги. При реално внедряване е необходимо допълнително конфигуриране на инфраструктурата, защитата, архивирането и мониторинга.

Въпреки тези ограничения, системата предоставя стабилна основа за изграждане на практическо решение за управление на документи. Архитектурата ѝ позволява постепенно разширяване чрез добавяне на нови модули, подобряване на сигурността, интеграция с външни услуги и адаптиране към конкретните нужди на дадена организация.

2.6. Функционални изисквания към системата

Функционалните изисквания описват основните действия, които системата MyDrive трябва да предоставя на своите потребители. Те са пряко свързани с предназначението на платформата като уеб базирана система за съхранение, управление, споделяне и обработка на електронни документи.

Системата трябва да осигурява регистрация и автентикация на потребители чрез защитен механизъм за вход. След успешно влизане потребителят трябва да получава достъп до собствено файлово пространство, в което може да създава папки, да качва файлове, да ги изтегля, преименува, изтрива и организира според своите нужди.

Друго важно функционално изискване е възможността за споделяне на файлове и папки с други потребители или чрез публични връзки. При споделяне системата трябва да

позволява задаване на права за достъп, като например право само за преглед или право за редактиране. По този начин се осигурява по-добър контрол върху достъпа до информацията.

Системата трябва да поддържа съвместна работа върху документи в реално време. Това позволява на няколко потребители едновременно да редактират един и същ документ, като промените се синхронизират между участниците. Тази функционалност е особено важна за екипна работа, учебни проекти, административни процеси и документооборот в организационна среда.

MyDrive трябва да предоставя и възможност за електронно подписване на PDF документи. Чрез тази функционалност потребителите могат да подписват документи по електронен път, като се цели съответствие с изискванията на регламента eIDAS и стандарта PAdES. Това разширява приложимостта на системата в контекста на официални документи, договори и други файлове, изискващи удостоверяване на авторство и цялост.

Към функционалните изисквания се включва и управление на потребителски профили, абонаментни планове и квоти за съхранение. Системата трябва да следи използваното дисково пространство и да ограничава действията на потребителите според зададените параметри на съответния план.

Допълнително изискване е интеграцията на AI асистент, който подпомага потребителите при работа със системата. Асистентът може да бъде използван за улесняване на навигацията, предоставяне на помощна информация и подпомагане при изпълнение на определени действия в платформата.

2.7 Нефункционални изисквания към системата

Нефункционалните изисквания определят качествените характеристики на системата и начина, по който тя трябва да изпълнява своите функции. Те са от съществено значение за надеждността, сигурността, производителността и удобството при работа с MyDrive.

Едно от основните нефункционални изисквания е сигурността. Системата трябва да защитава потребителските данни чрез надеждна автентикация, авторизация и контрол на достъпа. Достъпът до файлове и папки трябва да бъде ограничен само до потребители, които имат съответните права. Комуникацията между отделните

компоненти на системата трябва да бъде защитена чрез криптирани канали, особено при предаване на чувствителна информация и файлово съдържание.

Друго важно изискване е надеждността. Платформата трябва да осигурява стабилна работа при нормално натоварване и да минимизира риска от загуба на данни. Съхраняваните файлове и метаданни трябва да бъдат управлявани по начин, който позволява тяхното коректно извличане, изтегляне и обработка.

Системата трябва да бъде мащабируема, така че при увеличаване на броя потребители, файлове и заявки да може да бъде разширявана без необходимост от цялостна промяна на архитектурата. Разделянето на приложението на отделни компоненти, като Spring Boot API, файлов сървис, база данни и клиентски интерфейс, подпомага бъдещото развитие и разгръщане на системата.

Производителността също е съществено нефункционално изискване. Основните операции като вход в системата, зареждане на файлове и папки, качване, изтегляне и споделяне на документи трябва да се изпълняват в разумно време. Потребителският интерфейс трябва да бъде достатъчно бърз и отзивчив, за да осигури удобна работа с платформата.

От гледна точка на използваемостта системата трябва да предлага интуитивен и лесен за разбиране интерфейс. Потребителите трябва да могат бързо да откриват основните функции, без да е необходимо предварително обучение. Това е важно, тъй като платформата е предназначена както за технически потребители, така и за хора без задълбочени познания в областта на информационните технологии.

Поддържаемостта е друго важно изискване. Кодът на системата трябва да бъде организиран в ясни слоеве и модули, което улеснява откриването на грешки, добавянето на нови функционалности и бъдещото развитие на проекта. Използването на утвърдени технологии като Spring Boot, Next.js, MySQL и Docker допринася за по-лесна поддръжка и по-добра съвместимост с различни среди.

В обобщение, функционалните изисквания определят какво трябва да може да извършва системата MyDrive, докато нефункционалните изисквания описват как тези функции трябва да бъдат реализирани по отношение на сигурност, надеждност, производителност, мащабируемост и удобство за потребителя. Двете групи изисквания

заедно формират основата за проектирането и реализацията на стабилна и практическа платформа за управление на електронни документи.

III. ПРОЕКТИРАНЕ НА БАЗАТА ДАННИ И АРХИТЕКТУРАТА НА СИСТЕМАТА

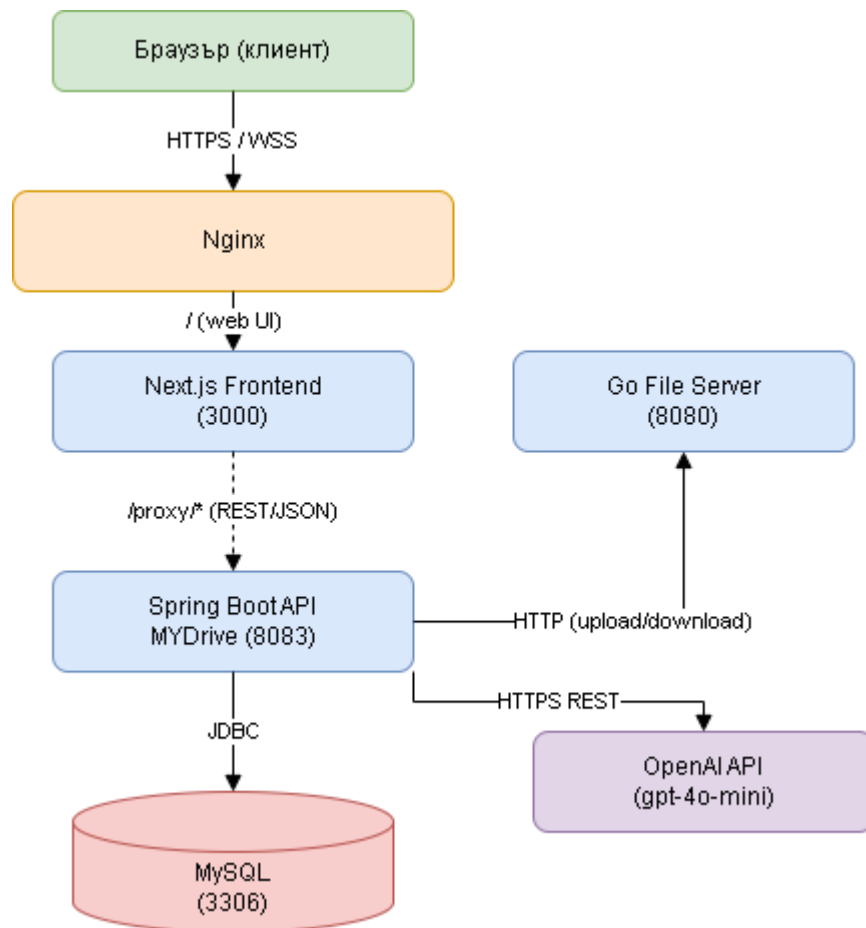
3.1. Системна архитектура - обзор

Системата MyDrive е реализирана като многослойно уеб приложение, следващо принципа за разделение на отговорностите (*Separation of Concerns*). Този подход позволява ясно разграничаване между потребителския интерфейс, бизнес логиката, управлението на данните и физическото съхранение на файловете, което улеснява поддръжката, разширяването и мащабирането на системата.

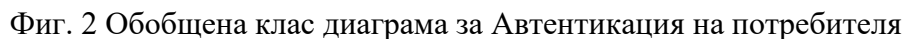
Архитектурата е изградена от няколко основни компонента, които комуникират помежду си чрез *HTTP* заявки и специализирани протоколи за обмен на данни. Бизнес логиката е реализирана в отделен *REST API* сървър, физическото съхранение на файловете е изнесено във файлов сървис, а потребителският интерфейс е реализиран като уеб приложение, достъпно чрез стандартен интернет браузър. Метаданните за потребителите, файловете и системните обекти се съхраняват в релационна база данни.

Таблица 1 Основни компоненти

Компонент	Технология	Роля	Порт
Java API (MYDrive)	Spring Boot 3.5, Java 21	Бизнес логика, автентикация, метаданни	8083
File Server	Go	Физическо съхранение и изтегляне на двоични файлове	8080
Web Frontend	Next.js 15, React 19, TypeScript	Потребителски интерфейс в браузъра	3000
MySQL	MySQL 8	Релационна база данни за метаданни	3306



Фиг. 1:Обща архитектура на приложението



3.2.1. Spring Boot API (MYDrive)

Компонентът предоставя *REST API* интерфейс за управление на потребители, файлове, папки, споделяния и абонаментни планове. Освен това реализира механизми за автентикация и авторизация чрез *JWT* токени, координира процесите по електронно подписване на документи и осъществява комуникация с външни услуги, включително *AI* асистента и файловия сървис.

16

- controllers – съдържа *REST* контролерите, които приемат и обработват *HTTP* заявки от клиентските приложения.
- services – реализира бизнес логиката на системата и координира взаимодействието между отделните компоненти.
- entities – съдържа *JPA* обектите, които моделират данните и се свързват с таблиците в релационната база данни.
- repositories – реализира слой за достъп до данните чрез *Spring Data JPA*.
- config – съдържа конфигурационните класове на приложението, включително настройките за сигурност и *WebSocket* комуникация.
- utils – включва помощни класове и обща функционалност, използвана от различните модули на системата.

Тази организация на кода осигурява добра разделимост на отговорностите, улеснява поддръжката на приложението и позволява бъдещо разширяване на функционалността.

3.2.2. Go File Server

Файловият сървис представлява самостоятелен компонент, реализиран на езика *Go*, чиято основна функция е физическото съхранение и предоставяне на файловото съдържание. Отделянето на файловете операции в независим сървис позволява бизнес логиката и управлението на метаданните да бъдат разграничени от процесите по съхранение и извличане на файлове, което улеснява поддръжката и бъдещото разширяване на системата.

Компонентът приема заявки за качване на файлове чрез *HTTP* интерфейс, съхранява файловете във файловата система на сървъра и генерира уникални *UUID*-базирани идентификатори за тяхното именуване. По този начин се предотвратяват конфликти между файлове с еднакви имена и се осигурява еднозначна идентификация на съхраняваното съдържание. В текущата реализация файловете се съхраняват във файловата система на сървъра, като при разгръщане чрез *Docker* се използва именуван *Docker volume*, гарантиращ запазването на данните независимо от жизнения цикъл на контейнерите.

Освен качването на файлове, сървисът предоставя функционалност за тяхното изтегляне и изтриване. Комуникацията с него се осъществява чрез основния *Spring Boot* компонент, който обработва потребителските заявки и съхранява съответните метаданни в базата данни. По този начин файловият сървис изпълнява ролята на

специализиран слой за управление на файловото съдържание, докато бизнес логиката остава концентрирана в основното приложение.

За комуникация между *Spring Boot API* и файловия сървис е избран *HTTP* протокол вместо *FTP*. Освен че *HTTP* се интегрира естествено с *REST* архитектурата на системата, той предоставя и по-добри възможности за защита на комуникацията чрез *TLS* криптиране. Класическият *FTP* протокол предава потребителските идентификационни данни и файловото съдържание в незашифриран вид, което създава предпоставки за прихващане и анализ на мрежовия трафик. Използването на *HTTPS* позволява криптиране на цялата комуникация между компонентите и значително намалява риска от неоторизиран достъп до предаваните данни.

3.2.3. Next.js Frontend

Потребителският интерфейс на системата е реализиран като едностранично уеб приложение, разработено с помощта на *Next.js* и *React*. Клиентската част осигурява достъп до всички функционалности на системата чрез стандартен интернет браузър и предоставя интуитивен интерфейс за работа с файлове, папки, споделяния и потребителски настройки.

При разработката е използван *TypeScript*, който осигурява статична проверка на типовете и улеснява поддръжката на кода. За изграждане на потребителския интерфейс е използвана библиотеката *Tailwind CSS*, която позволява създаването на адаптивен и последователен дизайн.

Комуникацията между клиентската част и сървърното приложение се осъществява чрез *HTTP* заявки към *REST API* интерфейса. За подобряване на сигурността и избягване на проблеми, свързани с *Cross-Origin Resource Sharing (CORS)*, заявките преминават през междинен прокси слой, реализиран в *Next.js*, който препраща заявките към основния *Spring Boot* сървър.

- Основните функционални области на потребителския интерфейс включват:
- • публична начална страница;
- • регистрация и автентикация на потребители;
- • управление на пароли и възстановяване на достъп;
- • навигация и управление на файлове и папки;
- • управление на споделянията и правата за достъп;
- • управление на абонаментни планове;

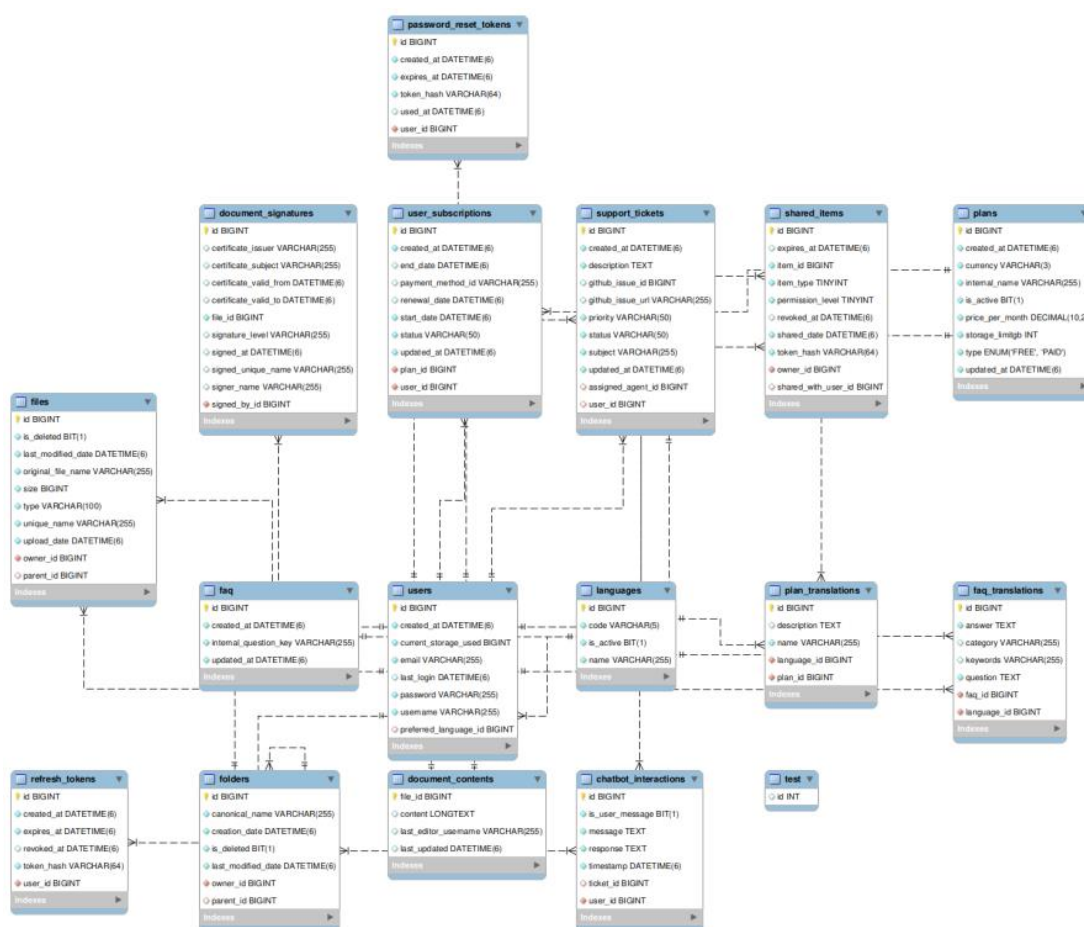
- настройки на потребителския профил;
- достъп до ресурси чрез публични връзки за споделяне.

Архитектурата на клиентското приложение е организирана по начин, който позволява лесно разширяване на функционалността и поддържане на добра производителност при работа с голям брой потребителски заявки.

3.3. Схема на базата данни

За съхранение на метаданните в системата MyDrive е използвана релационна база данни *MySQL 8*. Достъпът до данните се реализира чрез *Hibernate ORM* и спецификацията *Jakarta Persistence API (JPA)*, които осигуряват обектно-релационно преобразуване между програмните обекти и таблиците в базата данни.

Структурата на базата данни е проектирана така, че да поддържа управлението на потребители, файлове, папки, споделяния, абонаментни планове и електронни подписи. Основните таблици и техните атрибути са представени по-долу.



Фиг.3 Схема на базата данни

3.4. Комуникация между компонентите

Комуникацията между компонентите на решението MyDrive е организирана чрез ясно дефинирани интерфейси, които осигуряват разделение между потребителския интерфейс, бизнес логиката, съхранението на данни и външните услуги.

- **Браузър → Next.js** – потребителят взаимодейства със системата чрез уеб браузър. В производствена среда комуникацията се осъществява чрез *HTTPS*, а в среда за разработка може да се използва *HTTP*.
- **Next.js → Spring Boot API** – клиентското приложение изпраща *REST* заявки към проху слой, реализиран в *Next.js*. *Proху* сървърът препраща заявките към *Spring Boot API*, като по този начин се избягват проблеми, свързани с *Cross-Origin Resource Sharing (CORS)*, и се осигурява възможност за добавяне на допълнителна *middleware* логика.
- **Spring Boot API → MySQL** – достъпът до базата данни се осъществява чрез *Hibernate ORM* и *JDBC connection pool*. В базата данни се съхраняват метаданни за потребителите, файловете, папките, споделянията, абонаментните планове и електронните подписи.
- **Spring Boot API → Go File Server** – при операции по качване, изтегляне и управление на файлове *Spring Boot API* комуникира с файловия сървис чрез *HTTP* заявки. *Go File Server* отговаря за физическото съхранение на файловете във файловата система, докато *Spring Boot* управлява единствено техните метаданни.
- **Браузър ↔ Spring Boot API** – за осигуряване на съвместно редактиране на документи в реално време браузърът установява *WebSocket* връзка със *Spring Boot API* чрез *STOMP/SockJS* точка за достъп.
- **Spring Boot API → OpenAI API** – заявките към *AI* асистента се обработват от *Spring Boot API*, който осъществява комуникация с *OpenAI API* и връща генерираните отговори към клиентското приложение.

3.5. Сигурност и автентикация

Сигурността на системата е реализирана чрез *Spring Security* и *JWT (JSON Web Token)* механизъм за автентикация и авторизация. Архитектурата следва принципа на *stateless* комуникация, при който сървърът не съхранява информация за потребителските сесии

между отделните заявки. Вместо това необходимата информация за удостоверяване се съдържа в издадените токени.

Процесът на автентикация протича чрез използване на краткотрайни токени за достъп и дълготрайни *refresh* токени. Паролите на потребителите се съхраняват в базата данни под формата на *BCrypt* хешове, което предотвратява съхраняването на чувствителна информация в открит вид.

При успешно удостоверяване сървърът издава *JWT* access токен, подписан чрез алгоритъма *HMAC-SHA256*, както и *refresh* токен, използван за подновяване на достъпа без необходимост от повторно въвеждане на потребителските данни.

3.6. Инфраструктура - Docker и Nginx

3.6.1. Docker и Docker Compose

Системата MyDrive е проектирана за разгръщане чрез *Docker* и *Docker Compose*. Контейнеризацията осигурява изолирано изпълнение на отделните компоненти, улеснява управлението на зависимостите и гарантира идентична работна среда при разработка, тестване и експлоатация.

Използването на *Docker* предоставя следните предимства:

- **Изоляция на компонентите** – всеки компонент на системата се изпълнява в самостоятелен контейнер със собствен набор от зависимости.
- **Повторяемост на средата** – едни и същи *Docker* образи могат да бъдат използвани в различни среди без необходимост от допълнителна конфигурация.
- **Автоматизирано управление на зависимостите** – стартирането на отделните компоненти се координира чрез *Docker Compose*, като се осигурява коректна последователност на инициализация.
- **Персистентност на данните** – файловото хранилище и базата данни използват *Docker volume* механизми, които запазват информацията независимо от жизнения цикъл на контейнерите.

3.6.2. Nginx

В архитектурата на MyDrive *Nginx* изпълнява ролята на централизирана входна точка за всички клиентски заявки към системата. Разполагането на *reverse proxy* слой пред

приложните компоненти позволява централизирано управление на мрежовия трафик, повишава сигурността и улеснява бъдещото мащабиране на системата.

Основните функции на *Nginx* включват:

- *SSL/TLS* терминация – обработка на криптираните *HTTPS* връзки и препращане на заявките към вътрешните компоненти на системата.
- Маршрутизиране на заявките – пренасочване на *REST API* и публичните заявки към Spring Boot приложението и на останалите заявки към *Next.js* клиента.
- Поддръжка на *WebSocket* комуникация – осигуряване на коректно проксиране на *STOMP/SockJS* връзките, използвани за съвместно редактиране на документи в реално време.
- Повишаване на сигурността – добавяне на *HTTP* защитни заглавия и ограничаване на директния достъп до вътрешните сървиси.
- Автоматично пренасочване към *HTTPS* – всички незашифровани *HTTP* заявки се пренасочват към защитена *HTTPS* връзка.
- По този начин вътрешните компоненти на системата остават недостъпни директно от външната мрежа, а цялата комуникация преминава през единна контролирана точка за достъп.

IV. ОСОБЕНОСТИ НА ПРОГРАМНАТА РЕАЛИЗАЦИЯ

4.1. *Backend - Spring Boot*

4.1.1. Стартиране на приложението

Входната точка на сървърното приложение е класът `MyDriveApplication`, анотиран с `@SpringBootApplication`. При стартиране *Spring Boot* инициализира необходимите компоненти (*Spring Beans*), зарежда конфигурационните параметри и стартира вградения уеб сървър, чрез който приложението започва да приема клиентски заявки.

В процеса на инициализация *Hibernate ORM* установява връзка с базата данни и синхронизира обектния модел със схемата на базата данни съгласно зададената конфигурация. Това позволява автоматично създаване и актуализиране на необходимите таблици и връзки между тях.

Основните конфигурационни параметри на приложението се съхраняват във файла `src/main/resources/application.properties`. В него са дефинирани настройките за връзка с базата данни *MySQL*, параметрите за *JWT* автентикация, директорията за съхранение на файловете, *SMTP* настройките за изпращане на електронна поща, както и параметрите за достъп до *PKCS#12* хранилището, използвано при електронното подписване на документи.

4.1.2. Автентикация - *AuthController* и *AuthService*

Класът `AuthController`, разположен в пакета `controllers`, реализира *REST* интерфейсите, свързани с регистрацията, автентикацията и възстановяването на достъпа до потребителски профили. Бизнес логиката за тези операции е реализирана в класа `AuthService`, който координира взаимодействието между компонентите за сигурност, базата данни и услугата за електронна поща.

Основните методи на `AuthService` са:

- `register()` – проверява уникалността на потребителското име и имейл адреса, хешира паролата чрез *BCrypt* алгоритъм, създава нов потребител и автоматично го асоциира с безплатен абонаментен план.
- `login()` – извършва автентикация чрез *Spring Security AuthenticationManager* и при успешно удостоверяване генерира *JWT access token* с валидност един час и *refresh token* с валидност 30 дни.

- `refresh()` – валидира получения *refresh token*, проверява неговото състояние в базата данни и реализира механизма *token rotation* чрез генериране на нова двойка токени.
- `logout()` – деактивира активния *refresh token* и прекратява възможността за последващо подновяване на потребителската сесия.
- `forgotPassword()` и `resetPassword()` – реализират процеса по възстановяване на достъпа чрез изпращане на еднократен токен по електронна поща и задаване на нова парола след успешна валидация на токена.

```
// AuthController.java - фрагмент
@PostMapping("/login")
public AuthTokensResponse login(@Valid @RequestBody UserLoginRequest req) {
    return authService.login(req);
}

@PostMapping("/refresh")
public AuthTokensResponse refresh(@Valid @RequestBody RefreshRequest req) {
    return authService.refresh(req); // token rotation
}

@GetMapping("/me")
public UserResponse me(JwtAuthenticationToken auth) {
    Long userId = ((Number) auth.getTokenAttributes().get("uid")).longValue();
    return authService.getUserResponseById(userId);
}
```

Класът *JwtService* отговаря за генерирането и валидирането на *JWT* токените, използвани за удостоверяване на потребителите в системата. При успешно вписване се създава токен, съдържащ основна информация за идентификация на потребителя под формата на *claim*-ове, включително електронен адрес и уникален идентификатор на потребителя.

Токенът се подписва чрез алгоритъма *HMAC-SHA256*, като използваният таен ключ се зарежда от конфигурационния файл *application.properties*. По този начин се гарантира целостта на съдържащата се в токена информация и се предотвратява неговото подправяне.

Валидирането на *JWT* токените се извършва автоматично от *Spring Security* при всяка защитена заявка. При успешно валидиране информацията за потребителя се зарежда в

контекста за сигурност (*Security Context*), което позволява последващо изпълнение на операции, изискващи удостоверен достъп.

4.1.3. Управление на файлове - **FileController** и **FileService**

FileController дефинира *CRUD* endpoint-и за файлове (*/api/files*). *FileService* управлява физическото запазване и метаданните в БД.

Основни операции:

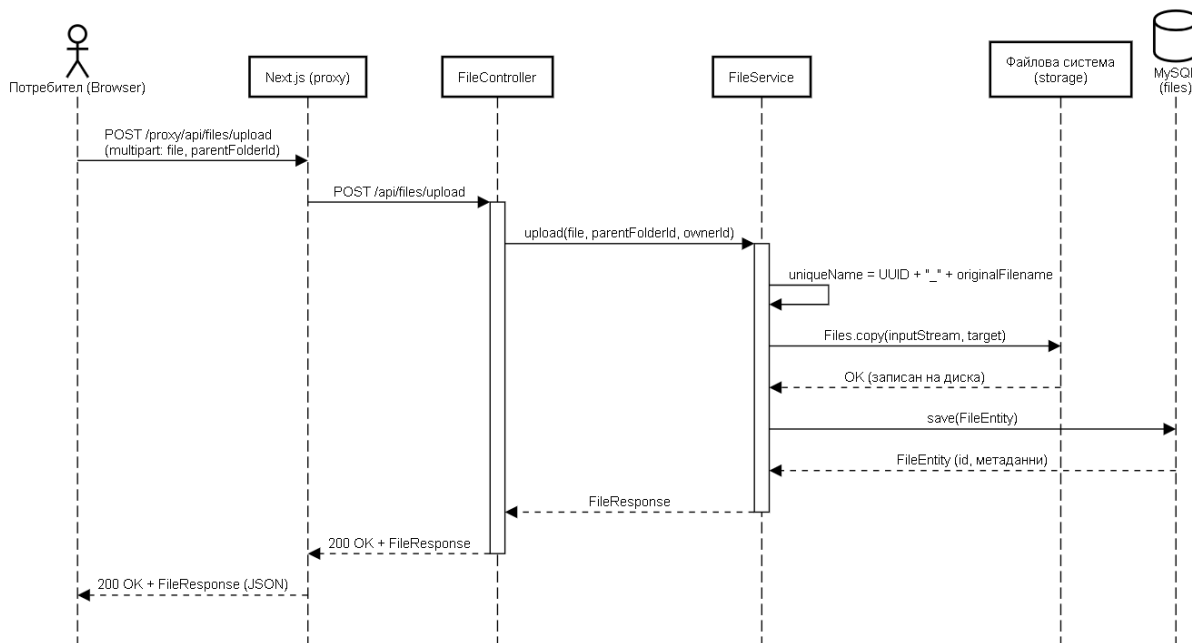
- `upload()` - приема *MultipartFile*, генерира *UUID*-базирано уникално наименование, записва файла на диска и създава запис в таблица *files*.
- `getDownloadResource()` - връща *PathResource* за изтегляне с оригиналното наименование и *MIME* тип.
- `update()` - преименува файла (само метаданни в БД, физическият файл не се мести).
- `move()` - мести файл в друга папка.
- `delete()` - soft delete (`isDeleted=true`) + физическо изтриване от диска.

```
// FileService.java - upload метод (фрагмент)
public FileResponse upload(MultipartFile file, Long parentFolderId, Long ownerId)
    throws IOException {
    String uniqueName = UUID.randomUUID() + "_" + file.getOriginalFilename();
    Path target = Path.of(storagePath, uniqueName);
    Files.copy(file.getInputStream(), target); // physical write

    FileEntity entity = FileEntity.builder()
        .originalFileName(file.getOriginalFilename())
        .uniqueName(uniqueName)
        .type(file.getContentType())
        .size(file.getSize())
        .owner(owner)
        .parent(parent)
        .isDeleted(false)
        .build();

    return FileResponse.from(fileRepository.save(entity));
}
```

4.1.3 Качване на файл (File Upload)



Фиг. 4 Процес по качване на файл

4.1.4. Spring Security конфигурация

Класът SecurityConfig реализира основната конфигурация на *Spring Security* и определя правилата за достъп до ресурсите в системата. Конфигурацията е изградена съгласно принципите на *stateless* автентикацията, при която сървърът не съхранява информация за потребителските сесии между отделните заявки.

Тъй като софтуерът използва *JWT* токени за удостоверяване на потребителите, защитата срещу *CSRF* атаки е изключена, тъй като не се използват *cookie-based* сесии. Всички заявки към защитените ресурси изискват валиден *JWT* токен, докато интерфейсите за регистрация, вход и достъп до публично споделени ресурси са достъпни без предварителна автентикация.

Основната конфигурация е реализирана по следния начин:

```

// SecurityConfig.java
http.csrf(AbstractHttpConfigurer::disable)
    .sessionManagement(sm -> sm
        .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/api/auth/**").permitAll()
        .requestMatchers("/public/shares/**").permitAll()
        .requestMatchers("/ws/**").permitAll()
        .anyRequest().authenticated())
    .oauth2ResourceServer(oauth -> oauth.jwt(Customizer.withDefaults()));
    
```

Конфигурацията определя кои ресурси са публично достъпни и кои изискват удостоверяване. Валидирането на *JWT* токените се извършва автоматично от *Spring Security OAuth2 Resource Server*, което намалява необходимостта от ръчна обработка на токените в приложния код.

4.2. File Server - Go

Файловият сървис, реализиран на езика *Go*, представлява самостоятелен микросървис, чиято основна функция е физическото съхранение и предоставяне на файлово съдържание. По този начин файловите операции са отделени от бизнес логиката на приложението, което улеснява поддръжката и бъдещото разширяване на разработката.

В текущата реализация са разработени два основни интерфейса:

- *POST /upload* – приема файлове чрез *multipart/form-data* заявка и ги съхранява във файловата система под уникално *UUID*-базирано наименование.
- *GET /download/{filename}* – предоставя достъп до съхранен файл чрез неговото уникално име и връща подходящите *HTTP* заглавия за коректно изтегляне от клиента.

Отделянето на файловото хранилище в самостоятелен микросървис позволява независимо мащабиране на компонентите на системата. По този начин натоварването, породено от операции по качване и изтегляне на файлове, не влияе пряко върху изпълнението на бизнес логиката в основното приложение.

Допълнително архитектурното решение позволява файловият сървис да бъде заменен с външно *S3*-съвместимо хранилище, като например *MinIO*, *Amazon S3* или *Cloudflare R2*, без съществени промени в сървърната логика на системата.

4.3. Frontend - Next.js

4.3.1. Структура и маршрутизиране

Фронтендът следва App Router модела на *Next.js* 15. Защитените страници са в директория (*protected*)/, като проверява наличието на *access token* - при липса пренаסочва към */login*. Публичните страници (*/*, */login*, */register*, */share/[token]*) са достъпни без автентикация.

4.3.2. AuthContext - глобално управление на автентикацията

AuthContext (contexts/AuthContext.tsx) управлява глобалното автентикационно състояние на приложението чрез *React Context API*. При зареждане прочита кешираните user и accessToken от localStorage и валидира сесията чрез GET /api/auth/me.

```
// AuthContext.tsx - инициализация при зареждане
useEffect(() => {
  const cached = localStorage.getItem("user");
  const token = localStorage.getItem("accessToken");
  if (cached && token) {
    setUser(JSON.parse(cached));
    refreshUser().finally(() => setLoading(false));
  } else {
    setLoading(false);
  }
}, [refreshUser]);
```

4.3.3. API клиент (lib/api.ts)

Централният API клиент (lib/api.ts) капсулира всички HTTP заявки в типизирани функции. Функцията request<T>() автоматично:

- Добавя Authorization: Bearer {accessToken} заглавие при всяка заявка.
- При получаване на 401 извиква refreshTokens() и повтаря заявката веднъж.
- При неуспешен refresh изчиства токените и пренасочва към /login.

Дефинирани API групи:

- auth - login, register, logout, me, forgotPassword, resetPassword, changePassword.
- filesApi - list, get, create, update, move, delete, upload (multipart), download.
- foldersApi - list, create, update, delete.
- sharesApi - list, create, revoke.
- publicSharesApi - get (без автентикация).
- chatApi - send (изпраща plain text, получава plain text отговор).
- documentsApi - getContent (последно съдържание на документ).
- signingApi - sign (*eIDAS PAdES*), getSignatures, downloadUrl.

4.3.4. DriveView - централен компонент на файловия мениджър

DriveView (components/drive/DriveView.tsx) е главният компонент за управление на файловата структура. Зарежда папките и файловете за текущото ниво, показва ги в грид изглед и управлява всички модали:

- CreateFolderModal - създаване на нова папка.
- UploadModal - качване на файл с multipart форма.
- RenameModal - преименуване на файл или папка.
- DeleteModal - потвърждение за изтриване.
- ShareModal - генериране на споделена връзка с настройки за достъп.
- MoveModal - местене на файл в друга папка.
- DocumentEditor - вграден реално-времеви текстов редактор.
- SignDocumentModal - eIDAS PAdES подписване на PDF.

4.3.5. Next.js Proxy Rewrite

В `next.config.ts` е конфигуриран `rewrite`: всички заявки към `/proxy/*` се препращат към `http://localhost:8083/:path*`. Това позволява фронтендът да изпраща *API* заявки относително (`/proxy/api/...`) без нужда от *CORS* конфигурация на бекенда.

```
// next.config.ts
const nextConfig = {
  async rewrites() {
    return [{
      source: "/proxy/:path*",
      destination: "http://localhost:8083/:path*",
    }];
  },
};
```

4.4. Реално-времево съвместно редактиране (WebSocket / STOMP)

MyDrive поддържа едновременно редактиране на текстови документи от множество потребители чрез *WebSocket* (*STOMP* протокол над *SockJS fallback*).

4.4.1. Сървърна конфигурация (WebSocketConfig)

`WebSocketConfig` конфигурира *STOMP* message broker: `/topic/*` за broadcast канали (публикуване към всички абониращи клиенти), `/app/*` за application destinations (съобщения, обработвани от контролери). *SockJS* е активиран като *fallback* за браузъри без *WebSocket* поддръжка.

4.4.2. CollaborationController

Класът `CollaborationController` обработва съобщенията, изпращани от потребителите по време на съвместно редактиране на документи. Съобщенията се приемат чрез *STOMP*

точката за достъп `@RequestMapping("/document/{fileId}/edit")`, след което актуалното съдържание на документа се записва в таблицата `document_contents`.

След успешно съхраняване на информацията съобщението се публикува към канала `@SendTo("/topic/document/{fileId}")`, към който са абонирани всички потребители, работещи върху съответния документ. По този начин всяка направена промяна се разпространява в реално време до всички активни участници в редакцията, което осигурява синхронизация на съдържанието между отделните клиентски приложения.

```
// CollaborationController.java
@RequestMapping("/document/{fileId}/edit")
@SendTo("/topic/document/{fileId}")
public DocumentEditMessage broadcastEdit(
    @DestinationVariable Long fileId,
    DocumentEditMessage message) {

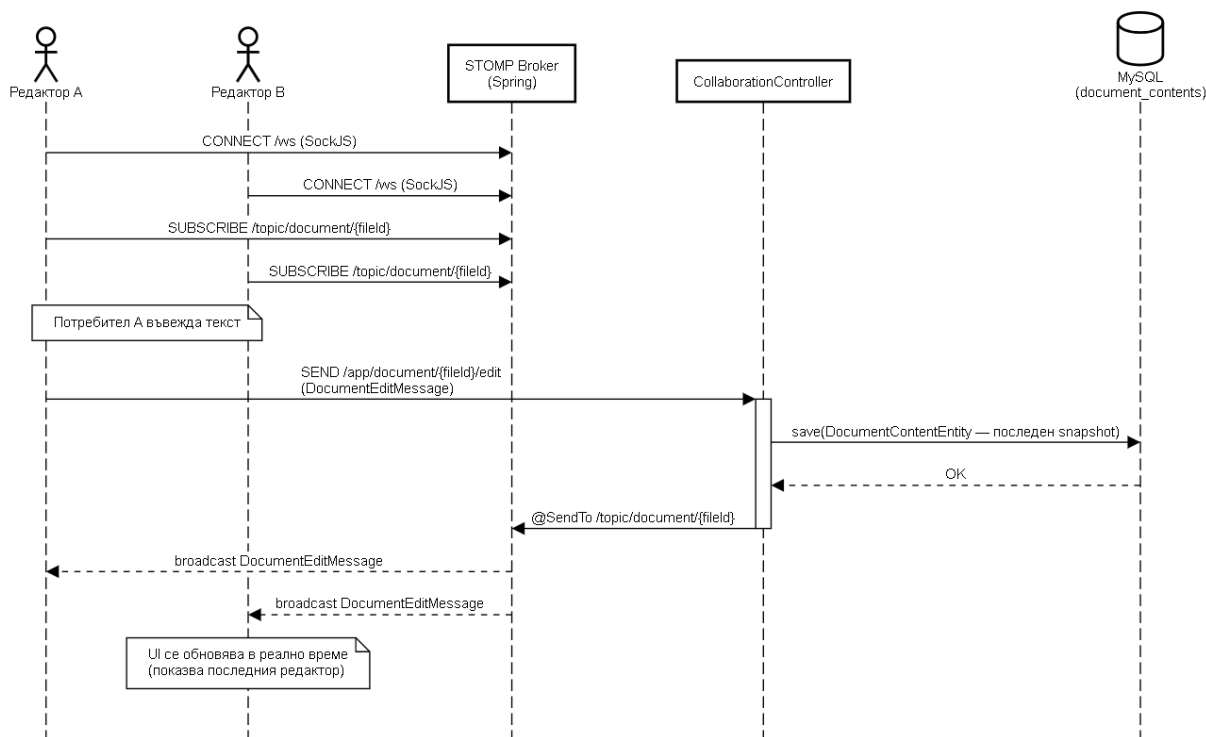
    // Persist latest snapshot in document_contents table
    DocumentContentEntity entity = contentRepository.findById(fileId)
        .orElseGet(() -> { /* create new */ });
    entity.setContent(message.content());
    entity.setLastEditorUsername(message.editorUsername());
    contentRepository.save(entity);

    return message; // broadcast to all subscribers on /topic/document/{fileId}
}
```

4.4.3. Клиентска страна (DocumentEditor)

`DocumentEditor` (`components/drive/DocumentEditor.tsx`) установява *STOMP/SockJS* връзка, абонира се за `/topic/document/{fileId}` и изпраща съобщения при всяка промяна в текстовото поле чрез `/app/document/{fileId}/edit`. Показва потребителското им на последния редактор в реално време.

4.4.3 Съвместно редактиране (WebSocket / STOMP)



Фиг. 5 Процес по синхронна работа на два потребителя върху един документ

4.5. Електронно подписване на документи (eIDAS PAdES)

MyDrive реализира *PAdES-B-B* (*PDF Advanced Electronic Signature - Baseline-B*) подписване, съвместимо с Регламент *eIDAS* (EU 910/2014), Член 26 - *Advanced Electronic Signature (AdES)*.

4.5.1. Автоматично генериране на demo keystore

При първо стартиране `DocumentSigningService.initKeystore()` проверява дали съществува keystore файл (.p12). Ако не - автоматично генерира:

- RSA-2048 key pair.
- Самоподписан X.509 v3 сертификат (Subject: CN=MyDrive Demo Signature, O=MyDrive, C=BG, валиден 10 години).
- PKCS#12 (.p12) файл, съдържащ private key и сертификата.

Пътят до keystore файла се конфигурира в `application.properties` (`app.signing.keystore-path`, `app.signing.keystore-password`, `app.signing.key-alias`).

4.5.2. Процес на PAdES-B-B подписване

Подписването на *PDF* файл протича в следните последователни стъпки:

- 1. Зареждане на оригиналния *PDF* с Apache PDFBox 3.x (Loader.loadPDF()).
- 2. Създаване на PDSignature обект с FILTER_ADOBE_PPKLITE и SUBFILTER_ADBE_PKCS7_DETACHED.
- 3. Генериране на *CMS (Cryptographic Message Syntax)* подпис с *SHA256withRSA*.
- 4. Вграждане на *PKCS#7* подписа в *PDF* чрез incrementalSave.
- 5. Записване на подписания файл на диска под ново уникално наименование (signed_{uuid}.pdf).
- 6. Съхраняване на метаданни за подписа в таблица document_signatures.

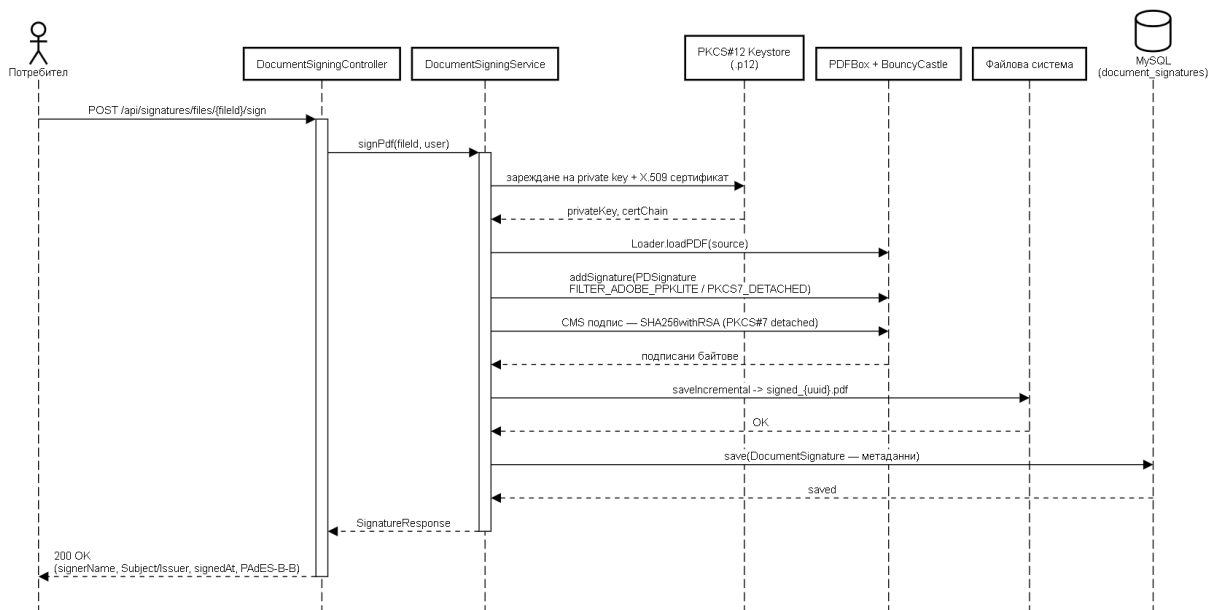
```
// DocumentSigningService.java - signPdf метод (фрагмент)
try (PDDocument doc = Loader.loadPDF(sourceFile.toFile())) {
    PDSignature signature = new PDSignature();
    signature.setFilter(PDSignature.FILTER_ADOBE_PPKLITE);
    signature.setSubFilter(PDSignature.SUBFILTER_ADBE_PKCS7_DETACHED);
    signature.setName(user.getUsername());
    signature.setReason("PAdES-B-B eIDAS Advanced Electronic Signature");

    doc.addSignature(signature, (InputStream content) -> {
        byte[] bytes = content.readAllBytes();
        CMSSignedDataGenerator gen = new CMSSignedDataGenerator();
        ContentSigner cs = new JcaContentSignerBuilder("SHA256withRSA")
            .setProvider("BC").build(privateKey);
        gen.addSignerInfoGenerator(...);
        gen.addCertificates(new JcaCertStore(List.of(chain)));
        CMSSignedData cmsData = gen.generate(
            new CMSPProcessableByteArray(bytes), false);
        return cmsData.getEncoded();
    }, opts);

    doc.saveIncremental(Files.newOutputStream(signedFile));
}
```

В производствена среда демо сертификатът се заменя с такъв от квалифициран доставчик на удостоверителни услуги (*QSP*) по смисъла на *eIDAS*, за постигане на *QES (Qualified Electronic Signature)* ниво.

4.5.2 PAdES-B-B подписване (eIDAS)



Фиг. 6 Процес за електронно подписване на документ

4.6. AI Чатбот

Системата MyDrive включва интегриран *AI* асистент, предназначен да подпомага потребителите при работа с платформата. Функционалността е реализирана чрез клиентски компонент, който предоставя интерфейс за обмен на съобщения между потребителя и сървърната част на приложението.

При изпращане на потребителска заявка клиентското приложение осъществява комуникация с *ChatController* чрез *REST* интерфейс (*POST /api/chat*). Контролерът делегира обработката към класа *OpenAIService*, който реализира комуникацията с *OpenAI Chat Completions API*.

В текущата реализация за генериране на отговори се използва моделът *GPT-4o mini*. След получаване на заявката от *OpenAI API* генерираният отговор се връща към клиентското приложение и се визуализира в потребителския интерфейс.

Интегрираният асистент е достъпен от всички защитени страници на системата и има за цел да подпомага потребителите при навигацията и използването на наличните функционалности. По този начин се намалява необходимостта от допълнителна документация и се подобрява цялостното потребителско преживяване.

```
// Конфигуриране на AI агент в приложението
@Configuration
public class OpenAIConfig {
```

```

@Value("${openai.api.key}")
private String apiKey;
@Bean
public WebClient openAIWebClient() {
    return WebClient.builder()
        .baseUrl("https://api.openai.com/v1")
        .defaultHeader("Authorization", "Bearer " + apiKey)
        .defaultHeader("Content-Type", "application/json")
        .build();
}

public ChatResponse chat(@RequestBody message){
    String knowledge =
    faqService.findAll().stream().collect(Collectors.joining(";"));

    String systemPrompt = "You are the support assistant for MyDrive, a cloud storage
    app. Answer ONLY using the knowledge base below. %s".formatted(knowledge);

    ChatRequest request = new ChatRequest(
        "gpt-4o-mini",
        List.of(
            new ChatMessage("system", systemPrompt),
            new ChatMessage("user", message)
        )
    );

    Return request.toResponse();
}
}

```

4.7. Платежна интеграция и абонаменти (Stripe)

Системата MyDrive поддържа три абонаментни плана – *Free*, *Pro* и *Business*, които определят максималния капацитет за съхранение и достъпните функционалности за потребителите.

За обработка на онлайн плащанията е интегрирана платформата *Stripe Checkout*. Използването на външна платежна услуга позволява чувствителните картови данни да бъдат обработвани директно от *Stripe*, без да преминават през сървърите на MyDrive и без да се съхраняват в локалната база данни. Това повишава сигурността на софтуера и намалява изискванията, свързани с обработката на платежна информация.

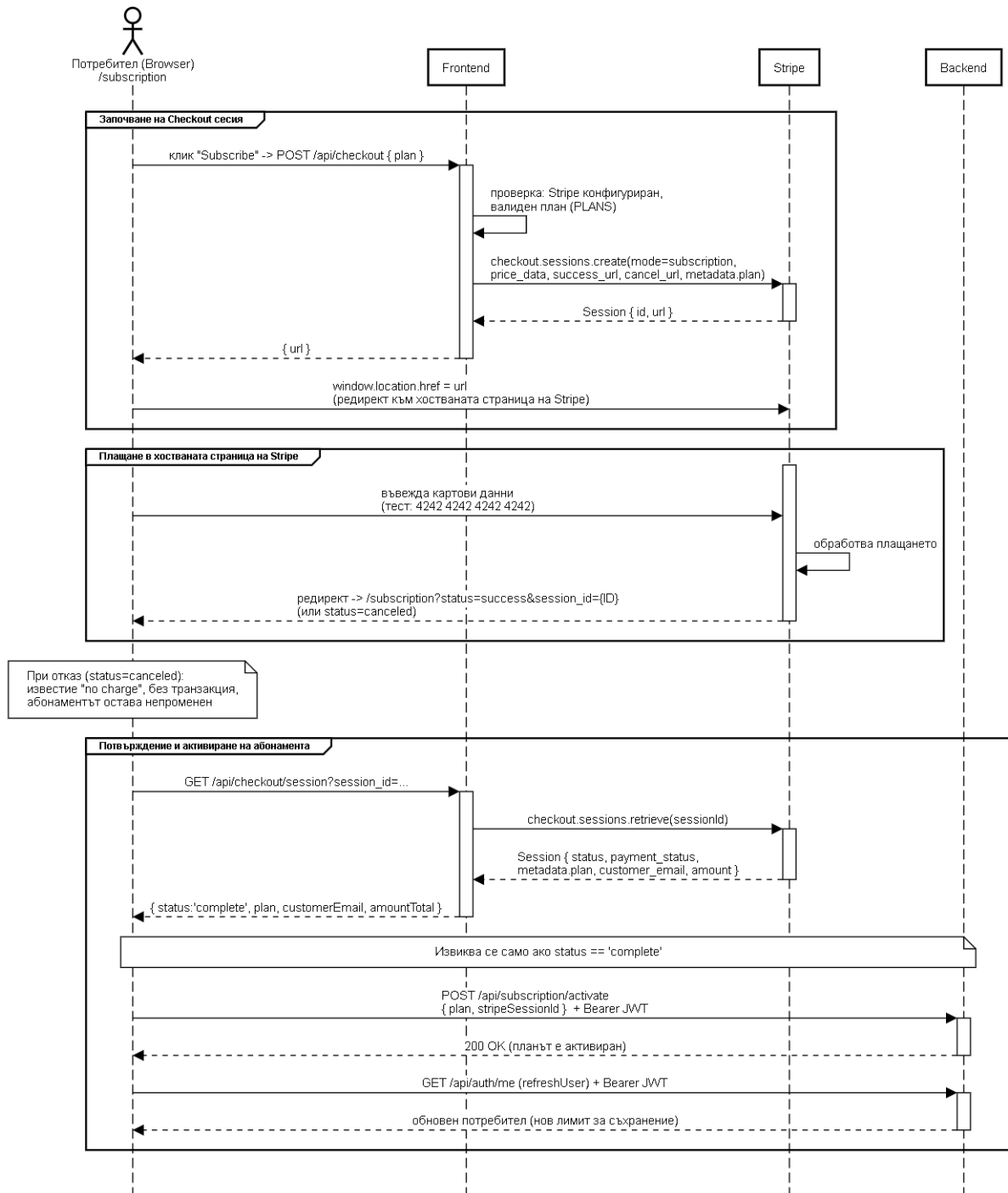
Процесът на активиране на платен абонамент протича в следната последователност:

- Потребителят избира желан абонаментен план.
- Клиентското приложение създава *Stripe Checkout* сесия чрез сървърен интерфейс.
- Потребителят се пренасочва към защитената платежна страница на Stripe и извършва плащането.
- След успешно приключване на плащането *Stripe* връща потребителя обратно към приложението.
- Клиентската част потвърждава статуса на плащането и изпраща заявка за активиране на избрания абонамент.
- Сървърната част актуализира информацията за абонамента в базата данни и активира съответния план за потребителя.

Системата поддържа еднозначна връзка между потребител и абонаментен план, като за всеки потребител може да съществува само един активен абонамент в даден момент. При промяна на плана съществуващата информация се актуализира, без да се създават допълнителни записи за активни абонаменти.

```
// Активиране на план след успешно плащане (SubscriptionService)
String internalName = PLAN_ALIASES.getOrDefault(planRef.toLowerCase(), planRef);
PlanEntity plan = planRepository
    .findByInternalNameAndIsActiveTrue(internalName)
    .orElseThrow(() -> new ResponseStatusException(BAD_REQUEST, ...));
sub.setPlanEntity(plan);
sub.setStatus("ACTIVE");
sub.setRenewalDate(LocalDate.now().plusMonths(1));
sub.setPaymentMethodId(stripeSessionId);
```

4.7 Плащане и активиране на абонамент (Stripe Checkout)



Фиг. 7 – Процес по заплащане на абонаментен план

4.8 Структура на база данни

Таблица 2: Структура на таблицата с потребителите

Колона	Тип	Описание
id	BIGINT PK AUTO	Уникален идентификатор
username	VARCHAR UNIQUE NOT NULL	Потребителско име (lowercase)

email	VARCHAR UNIQUE NOT NULL	Имейл адрес (lowercase)
password	VARCHAR NOT NULL	BCrypt хеш на паролата
createdAt	DATETIME NOT NULL	Дата на регистрация (автоматична)
lastLogin	DATETIME	Последен успешен вход
currentStorageUsed	BIGINT NOT NULL	Изразходвано пространство в байтове
preferredLanguageId	BIGINT FK->languages	Предпочитан език (опционален)

Таблица 3: Структура на таблицата с абонаментни планове

Колона	Тип	Описание
id	BIGINT PK AUTO	Идентификатор
internalName	VARCHAR UNIQUE NOT NULL	Код на плана: FREE_PLAN, BASIC_PLAN
type	ENUM(FREE,PAID)	Тип на плана
storageLimitGB	INT NOT NULL	Лимит за съхранение в GB
pricePerMonth	DECIMAL(10,2) NOT NULL	Месечна цена
currency	VARCHAR(3) NOT NULL	Валута (напр. BGN)
isActive	BOOLEAN NOT NULL	Дали планът е активен

Таблица 4: Структура на таблицата с абониращи потребители

Колона	Тип	Описание
id	BIGINT PK AUTO	Идентификатор
user_id	BIGINT FK->users UNIQUE	Потребител
plan_id	BIGINT FK->plans	Избран план
startDate	DATETIME NOT NULL	Начало на абонамента
endDate	DATETIME NULL	Край (null = активен)
status	VARCHAR(50) NOT NULL	ACTIVE / EXPIRED / CANCELLED
renewalDate	DATETIME NULL	Дата на следващо подновяване
paymentMethodId	VARCHAR NULL	ID на Stripe Checkout сесията, потвърдила плащането

Таблица 5: Структура на таблицата за съхранени папки

Колона	Тип	Описание
id	BIGINT PK AUTO	Идентификатор
canonicalName	VARCHAR NOT NULL	Базово наименование на папката
owner_id	BIGINT FK->users NOT NULL	Собственик
parent_id	BIGINT FK->folders NULL	Родителска папка (null = root ниво)
isDeleted	BOOLEAN NOT NULL	Soft delete флаг
creationDate	DATETIME NOT NULL	Дата на създаване (автоматична)

lastModifiedDate	DATETIME NOT NULL	Дата на последна промяна (автоматична)
------------------	-------------------	--

Таблица 6: Структура на таблицата за съхранени файлове

Колона	Тип	Описание
id	BIGINT PK AUTO	Идентификатор
originalFileName	VARCHAR NOT NULL	Оригинално наименование на файла
uniqueName	VARCHAR UNIQUE NOT NULL	UUID-базирано уникално физическо наименование
type	VARCHAR(100) NOT NULL	MIME тип (напр. application/pdf)
size	BIGINT NOT NULL	Размер в байтове
owner_id	BIGINT FK->users NOT NULL	Собственик
parent_id	BIGINT FK->folders NULL	Папка (null = root)
isDeleted	BOOLEAN NOT NULL	Soft delete флаг
uploadDate	DATETIME NOT NULL	Дата на качване (автоматична)
lastModifiedDate	DATETIME NOT NULL	Дата на последна промяна

Таблица 7: Структура на таблицата за споделени ресурси

Колона	Тип	Описание
id	BIGINT PK AUTO	Идентификатор
itemId	BIGINT NOT NULL	ID на споделения файл/папка
itemType	ENUM(FILE,FOLDER)	Тип на елемента
owner_id	BIGINT FK->users NOT NULL	Потребителят, иницирал споделянето
shared_with_user_id	BIGINT FK->users NULL	Потребител (null = публична връзка)
permissionLevel	ENUM(VIEW,EDIT,DELETE)	Ниво на достъп
tokenHash	VARCHAR(64) UNIQUE NOT NULL	SHA-256 хеш на токена
expiresAt	DATETIME NULL	Дата на изтичане (null = безсрочна)
revokedAt	DATETIME NULL	Дата на оттегляне (null = активно)
sharedDate	DATETIME NOT NULL	Дата на споделяне (автоматична)

Таблица 8: Структура на таблица за подписани документи

Колона	Тип	Описание
id	BIGINT PK AUTO	Идентификатор
fileId	BIGINT FK->files NOT NULL	Подписан PDF файл
signed_by	BIGINT FK->users NOT NULL	Потребителят, положил подписа
signerName	VARCHAR NOT NULL	Потребителско им по времето на подписване
certificateSubject	VARCHAR	Subject на X.509 сертификата

certificateIssuer	VARCHAR	Issuer на сертификата
certificateValidFrom	DATETIME	Начало на валидност на сертификата
certificateValidTo	DATETIME	Край на валидност на сертификата
signatureLevel	VARCHAR	Ниво на подписа (PAdES-B-B)
signedUniqueName	VARCHAR	Уникално наименование на подписания PDF файл
signedAt	DATETIME	Момент на подписване

4.9 Тестване на системата

За проверка на коректността и надеждността на разработената система са създадени набор от автоматизирани модулни тестове. Основната цел на тестовете е да валидират бизнес логиката на приложението и да гарантират коректното поведение на системата както при стандартни, така и при гранични и некоректни входни данни.

Тестовото покритие обхваща ключови компоненти на бекенд приложението, включително модулите за автентикация, управление на пароли, управление на папки, абонаментни планове и помощни класове. Особено внимание е отделено на обработката на грешки, валидирането на входните данни и проверката на изключителни ситуации.

4.9.1. Подход и инструменти

Тестовете следват подхода на изолирано модулно тестване, при който всеки компонент се проверява независимо от останалите части на приложението. Външните зависимости, като репозитории, услуги за изпращане на електронна поща и компоненти за криптиране, се заместват с *mock* обекти. Това позволява тестовете да бъдат бързи, предвидими и независими от външна инфраструктура.

За реализиране на тестовете са използвани следните технологии:

- **JUnit 5 (Jupiter)** – рамка за дефиниране и изпълнение на тестовите сценарии.
- **Mockito** – библиотека за създаване на *mock* обекти и проверка на взаимодействията между компонентите.
- **AssertJ** – библиотека за изграждане на четими и изразителни проверки на очакваните резултати.
- **spring-boot-starter-test** – стандартният тестов пакет на *Spring Boot*, включващ необходимите зависимости за модулно и интеграционно тестване, както и тест за успешно зареждане на *Spring* контекста.

4.9.2. Покрити модули и сценарии

Следната таблица обобщава тестовите класове, тестваните компоненти и броя на покритите сценарии (тестови методи):

Таблица 9: Обобщение за разработените тестове

Тестов клас	Тестван компонент	Брой тестове	Основни сценарии
AuthServiceTest	AuthService	20	Вход с email/потребителско име, ротация и отнемане на refresh токени, регистрация, конфликти, смяна на език
PasswordServiceTest	PasswordService	9	Забравена/нулирана/сменена парола, изтекли и вече използвани токени
FolderServiceTest	FolderService	17	Създаване, преименуване, преместване (вкл. в собствен наследник), soft delete, дублирани имена
SubscriptionServiceTest	SubscriptionService	8	Активни планове, активиране/смяна на абонамент, псевдоними на планове, неизвестни планове
SupportTicketServiceTest	SupportTicketService	8	Създаване на тикети, приоритети по подразбиране, контрол на достъпа
FaqServiceTest	FaqService	7	Локализация на често задавани въпроси, fallback към английски, сортиране
JwtServiceTest	JwtService	4	Генериране на JWT, subject/uid claim, срок на валидност, подпис
UserMapperTest	UserMapper	5	Мапиране entity<->DTO, разрешаване на лимит и език от активния план
GeneralUtilsTest	GeneralUtils	2	Извличане на потребителско ID от JWT в security контекста
TokenUtilTest	TokenUtil	6	Генериране на ораque токени и SHA-256 хеширане
MyDriveApplicationTests	Spring контекст	1	Успешно зареждане на приложния контекст

4.9.3. Примерен модулен тест

Даден е пример, който представя модулен тест на метода `activatePlan()` от класа `SubscriptionService`. Тестът следва утвърдения модел *Arrange-Act-Assert*, при който първоначално се подготвят необходимите обекти и входни данни, след което се извиква тестваният метод и накрая се проверява коректността на получения резултат.

В разглеждания сценарий се симулира активиране на абонаментен план от тип „Pro“ за потребител, който все още няма активен абонамент. Чрез *Mockito* се конфигурира поведението на използваните репозитории, а посредством *ArgumentCaptor* се прихваща обектът, записан в базата данни.

Тестът проверява дали публичният идентификатор на плана е коректно преобразуван към вътрешното му представяне, дали е създаден нов запис за абонамент и дали идентификаторът на Stripe сесията е съхранен правилно. Успешното преминаване на теста потвърждава коректната работа на бизнес логиката при активиране на нов абонамент.

```
@Test
void activatePlan_mapsAliasAndCreatesSubscriptionWhenNoneExists() {
    PlanEntity pro = plan(2L, "PRO_PLAN", true);
    when(userRepository.findById(1L)).thenReturn(Optional.of(user));
    when(planRepository.findByIdInternalNameAndIsActiveTrue("PRO_PLAN"))
        .thenReturn(Optional.of(pro));
    when(userSubscriptionRepository.save(any()))
        .thenAnswer(i -> i.getArgument(0));

    UserSubscriptionResponse res =
        subscriptionService.activatePlan(1L, "pro", "sess_123");

    assertThat(res.status()).isEqualTo("ACTIVE");
    ArgumentCaptor<UserSubscriptionEntity> captor =
        ArgumentCaptor.forClass(UserSubscriptionEntity.class);
    verify(userSubscriptionRepository).save(captor.capture());
    assertThat(captor.getValue().getPlanEntity()).isEqualTo(pro);
    assertThat(captor.getValue().getPaymentMethodId()).isEqualTo("sess_123");
}
```

V. РЪКОВОДСТВО НА ПОТРЕБИТЕЛЯ

Настоящото ръководство описва стъпка по стъпка основните операции в системата MyDrive. За достъп е необходим модерен браузър (*Chrome, Firefox, Edge, Safari*) и активна интернет връзка.

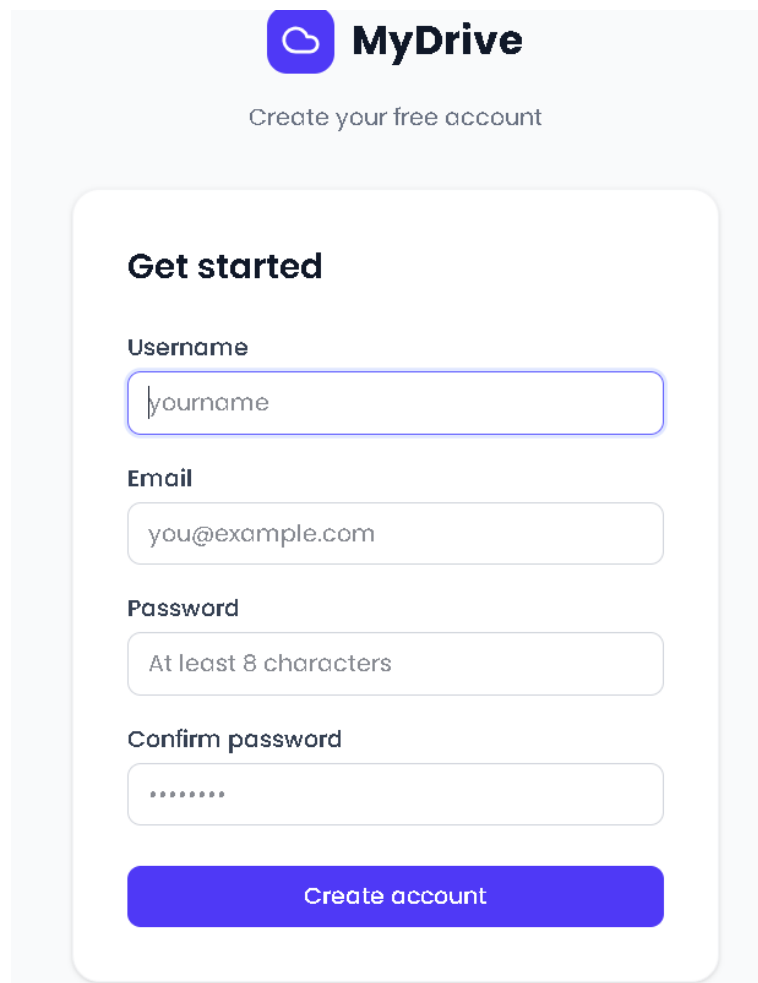
5.1. Регистрация и вход в системата

5.1.1. Регистрация на нов акаунт

За да създадете нов акаунт в MyDrive:

- Отворете началната страница на MyDrive в браузъра.
- Натиснете бутона Регистрация или отидете директно на /register.
- Попълнете полетата: Потребителско им, Емейл адрес и Парола.
- Натиснете "Register".
- При успех ще бъдете автоматично вписани и пренасочени към файловия мениджър (/drive).

Забележка: Потребителското им и имейлът трябва да са уникални в рамките на продукта. При регистрация автоматично се активира безплатен план с лимитирано пространство.

The image shows a registration form for 'MyDrive'. At the top, there is a blue cloud icon followed by the text 'MyDrive'. Below this, it says 'Create your free account'. The form itself is titled 'Get started' and contains four input fields: 'Username' with the placeholder 'yourname', 'Email' with the placeholder 'you@example.com', 'Password' with the placeholder 'At least 8 characters', and 'Confirm password' with a placeholder of seven dots. At the bottom of the form is a blue button labeled 'Create account'.

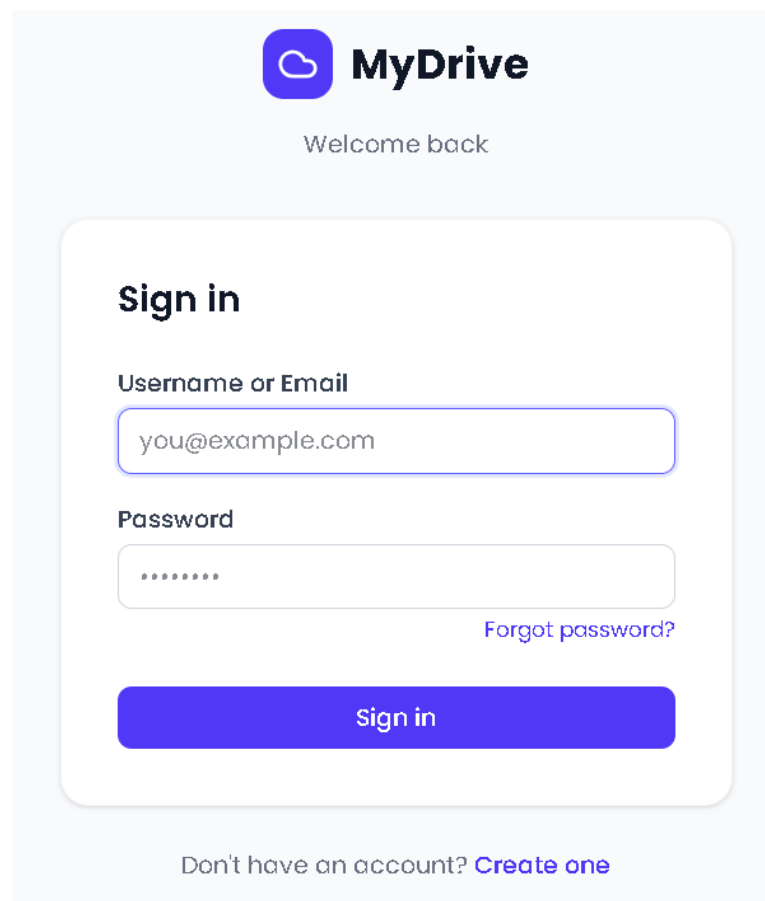
Фиг. 8 Страница за създаване на профил

5.1.2. Вход в системата

За вход в съществуващ акаунт:

- Отидете на /login.
- Въведете потребителско им или имейл и паролата си.
- Натиснете "Login".
- Системата ще ви пренасочи към файловия мениджър (/drive).

Ако сте забравили паролата си, натиснете "Forgot password" и следвайте инструкциите. Ще получите имейл с връзка за нулиране, валидна 15 минути.



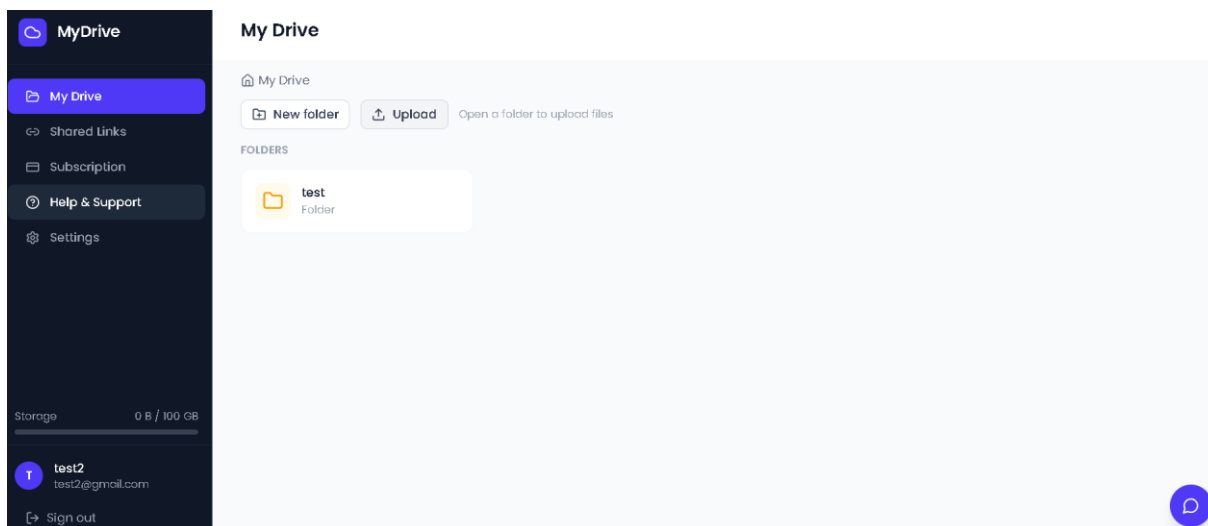
The image shows a login page for 'MyDrive'. At the top, there is a blue cloud icon followed by the text 'MyDrive'. Below this, it says 'Welcome back'. The main content is a white rounded rectangle with a 'Sign in' heading. It contains two input fields: 'Username or Email' with the placeholder 'you@example.com' and 'Password' with masked characters. A 'Forgot password?' link is to the right of the password field. A blue 'Sign in' button is at the bottom of the form. Below the form, there is a link: 'Don't have an account? [Create one](#)'.

Фиг. 9 Страница за вход

5.2. Управление на папки и файлове

5.2.1. Навигация

След вход попадате в главния изглед на MyDrive (/drive), показващ всички ваши root папки. Натиснете върху папка за навигация в нея. В горната part на страницата се показва breadcrumb навигация за лесно придвижване назад.



Фиг. 10 Начална страница на приложението

5.2.2. Създаване на нова папка

За да създадете нова папка:

- Натиснете бутона "New folder" (иконата с папка+) в лентата с инструменти.
- Въведете им на папката в диалоговия прозорец.
- Натиснете "Create".
- Новата папка ще се появи незабавно в текущия изглед.

5.2.3. Преименуване

Задръжте курсора над папка или файл и натиснете иконата за опции (три точки). Изберете "Rename", въведете новото наименование и потвърдете.

5.2.4. Изтриване

Изберете "Delete" от контекстното меню. Ще бъдете помолени за потвърждение. Изтритите файлове се маркират като изтрити (soft delete) и физически се премахват от диска.

5.2.5. Местене на файл

От контекстното меню на файл изберете "Move". В диалоговия прозорец изберете целевата папка от списъка. Натиснете "Move" за потвърждение.

5.3. Качване и изтегляне на файлове

5.3.1. Качване на файл

Файловете могат да се качват само вътре в папка (не на root ниво). Влезте в желаната папка и:

- Натиснете бутона "Upload" в лентата с инструменти.
- В диалоговия прозорец изберете файла от вашия компютър.
- Натиснете "Upload" за потвърждение.
- При успех файлът ще се появи в текущата папка.

Поддържаните формати са всички *MIME* типове. Максималният капацитет зависи от абонаментния план.

5.3.2. Изтегляне на файл

Натиснете иконата за опции (три точки) на желания файл и изберете "Download". Файлът ще бъде изтеглен с оригиналното си наименование.

5.4. Споделяне на файлове и папки

MyDrive позволява споделяне на файлове и папки чрез генерирани токен-връзки. Споделянето може да е с определен краен срок и ниво на достъп (VIEW или EDIT).

5.4.1. Създаване на споделена връзка

- Натиснете "Share" от контекстното меню на файл или папка.
- В модалния прозорец изберете: Ниво на достъп (VIEW или EDIT) и (опционално) дата на изтичане.
- Натиснете "Create share link".
- Копирайте генерирания URL и го изпратете на получателя.

5.4.2. Управление на споделяния

На страница /shares можете да видите всички ваши активни и оттеглени споделяния. За всяко споделяне се показват: тип (FILE/FOLDER), ниво на достъп, дата на създаване, дата на изтичане и статус.

За да оттеглите споделяне, натиснете "Revoke" до желаното споделяне. Оттеглените връзки спират да работят незабавно.

5.4.3. Достъп до споделена връзка

Получателят на споделена връзка може да отвори файла/папката на `/share/{token}` без нужда от акаунт в системата (публичен достъп). Показват се: тип на елемента, ниво на достъп и дата на изтичане.

5.5. Съвместно редактиране на документи

MyDrive поддържа реално-времево съвместно редактиране на текстови документи от множество потребители едновременно чрез *WebSocket* технология.

5.5.1. Отваряне на документ за редактиране

- Намерете текстов файл в папката.
- Натиснете "Edit" от контекстното меню.
- Ще се отвори вграденият редактор (*DocumentEditor*).

5.5.2. Работа в редактора

Редакторът показва текущото съдържание на документа. Всяка промяна, която направите, незабавно се синхронизира с всички потребители, отворили същия документ в момента. В горната part на редактора се вижда потребителското им на последния редактор.

При затваряне на редактора съдържанието се запазва автоматично. При следващо отваряне се зарежда последното записано съдържание.

5.6. Електронно подписване на PDF

MyDrive поддържа *PAdES-B-B* електронен подпис за *PDF* документи, съвместим с европейския регламент *eIDAS*.

5.6.1. Подписване на PDF документ

- Намерете *PDF* файл в папката.
- Натиснете "Sign" от контекстното меню.
- В модалния прозорец (*SignDocumentModal*) ще видите информация за файла.
- Натиснете "Sign document" за потвърждение.
- Системата ще генерира подписан *PDF* файл и ще покаже детайлите за подписа.

5.6.2. Преглед и изтегляне на подписания документ

След успешно подписване в модала ще видите информация за подписа: подписващ (signerName), сертификат Subject/Issuer, дата на подписване и ниво на подписа (PAdES-B-B). Можете да изтеглите подписания *PDF* чрез бутона "Download signed PDF".

Забележка: Само *PDF* файлове могат да се подписват. За другите формати опцията "Sign" не е налична.

5.7. AI Асистент

На всяка защитена страница в долния десен ъгъл се показва иконата на AI асистента. Натиснете я, за да отворите чат прозореца.

Можете да задавате въпроси на естествен език, например:

- "Как да споделя папка с колега?"
- "Как работи електронното подписване?"
- "Какво е PAdES?"

AI асистентът е базиран на *GPT-4o mini* и може да отговаря на широк спектър от въпроси, свързани с ползването на платформата.

5.8. Управление на профила и абонамента

Отидете на /settings за достъп до настройките на профила.

5.8.1. Промяна на парола

- В секция Security въведете текущата парола и новата парола.
- Натиснете "Change password".
- Паролата ще бъде сменена незабавно.

5.8.2. Информация за абонамента

В секция Subscription се показва текущият абонаментен план, лимитът за съхранение и изразходваното пространство чрез визуална лента (*StorageBar*).

5.8.3. Смяна на план и плащане

- Отворете страницата /subscription, за да разгледате наличните планове (Pro и Business) с техните цени и характеристики.

- Натиснете "*Subscribe*" под желания план - ще бъдете пренасочени към защитената платежна страница на Stripe.
- Въведете картовите си данни.
- След успешно плащане се връщате автоматично в MyDrive, новият план се активира незабавно и лимитът за съхранение се обновява.
- Ако откажете плащането, не се извършва никаква транзакция и абонаментът остава непроменен.

VI.ЗАКЛЮЧЕНИЕ

Настоящата дипломна работа представя проектирането и реализацията на MyDrive - пълнофункционална, локално внедрена облачна платформа за управление на документи. Системата обединява съвременни технологии в кохерентна архитектура: *Spring Boot 3* с *JWT* автентикация и *eIDAS PAdES* подписване, *Next.js 15* с *React 19* за потребителския интерфейс, *MySQL* за релационно съхранение на метаданни, *STOMP/WebSocket* за реално-времево сътрудничество и *AI* асистент, базиран на *GPT-4o mini*.

Инфраструктурата е проектирана с *Docker Compose* и *Nginx*, осигурявайки изолация на компонентите, повторяемост на средата и *SSL* терминация. Архитектурата с разделени отговорности позволява независимо мащабиране на бекенда, файловия сървър и фронтенда.

Механизмът за сигурност следва добрите практики: *BCrypt* за хеширане на пароли, *stateless JWT* автентикация с *token rotation*, *SHA-256* хеширане на *refresh* и *share* токени преди запис в БД, *Spring Security* с *OAuth2 Resource Server* за валидация на *JWT*. Монетизацията е реализирана чрез интеграция със *Stripe Checkout*, при която картови данни не се съхраняват в системата.

Надеждността на бизнес логиката е подсигурана чрез набор от автоматизирани модулни тестове (*JUnit 5*, *Mockito*, *AssertJ*), покриващи критичните услуги на бекенда - автентикация, управление на пароли, папки и абонаменти - с акцент върху граничните случаи и обработката на грешки.

VII. СПИСЪК НА ИЗПОЛЗВАНИТЕ ЛИТЕРАТУРНИ ИЗТОЧНИЦИ

- [1] Регламент (ЕС) № 910/2014 на Европейския парламент и на Съвета относно електронната идентификация и удостоверителните услуги (eIDAS). Официален вестник на ЕС, 2014.
- [2] ETSI EN 319 142-1. Electronic Signatures and Infrastructures (ESI); PAdES digital signatures; Part 1: Building blocks and PAdES baseline signatures.
- [3] Spring Boot Reference Documentation. Pivotal/VMware. <https://docs.spring.io/spring-boot/>
- [4] Spring Security Reference Documentation. <https://docs.spring.io/spring-security/reference/>
- [5] Next.js Documentation. Vercel. <https://nextjs.org/docs>
- [6] React Documentation. Meta Open Source. <https://react.dev/>
- [7] MySQL 8.0 Reference Manual. Oracle Corporation. <https://dev.mysql.com/doc/refman/8.0/en/>
- [8] Apache PDFBox - A Java PDF Library. The Apache Software Foundation. <https://pdfbox.apache.org/>
- [9] The Legion of the Bouncy Castle Java Cryptography APIs. <https://www.bouncycastle.org/>
- [10] Jones, M., Bradley, J., Sakimura, N. RFC 7519: JSON Web Token (JWT). IETF, 2015.
- [11] STOMP Protocol Specification, Version 1.2. <https://stomp.github.io/>
- [12] OpenAI API Reference. OpenAI. <https://platform.openai.com/docs/api-reference>
- [13] Stripe API Reference & Stripe Checkout Documentation. Stripe, Inc. <https://stripe.com/docs/api>
- [14] JUnit 5 User Guide. <https://junit.org/junit5/docs/current/user-guide/>
- [15] Mockito Documentation. <https://site.mockito.org/>
- [16] Docker Documentation. Docker Inc. <https://docs.docker.com/>
- [17] nginx documentation. <https://nginx.org/en/docs/>
- [18] The Go Programming Language Documentation. <https://go.dev/doc/>

VIII. Приложение

Адрес за достъп на кода на приложението: <https://github.com/Tirajiqta/MyDrive>